

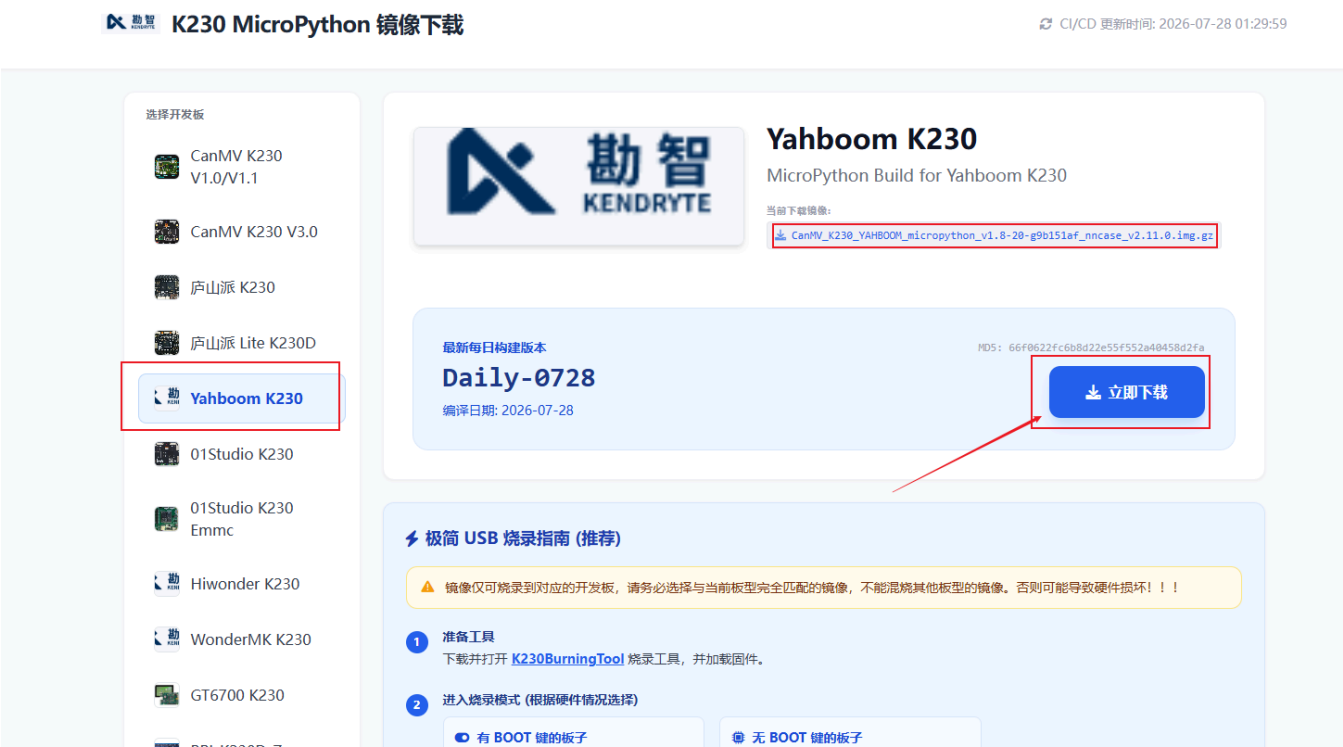
Image Blur Filtering

Image Blur Filtering

- Preface
- Example Results
- Principle Explanation
 - Why Filtering is Needed
- Code Overview
 - Initialize Camera and Display Module
 - Create Sharpening Kernel and Set Mode
 - Execute Filtering
 - Write Back to Original Frame and Display
 - Garbage Collection and Resource Release
- Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Jianan website https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/



Example Results

Run the example code for this section [10.camera filter blur.py](#).

This section demonstrates Gaussian, mean, median, bilateral, box filtering, and custom sharpening kernels on a 640×480 live image. Changing the mode allows you to compare the visual effects and frame rates of different filtering methods.

Principle Explanation

Why Filtering is Needed

Image filtering computes new pixel values from neighboring pixels, which can be used for denoising, smoothing, edge preservation, or sharpening. Different methods have varying effects on noise types, edges, and computation load.

Method	Key Feature	Common Uses
Gaussian Filter	Closer to center means higher weight	Random noise, edge detection preprocessing
Mean Filter	Equal weight averaging of neighbors	Fast smoothing
Median Filter	Takes median of neighbors	Salt and pepper noise
Bilateral Filter	Considers both spatial distance and color difference	Edge-preserving denoising
Box Filter	Rectangular neighbor sum or average	Fast local statistics
Custom Convolution	Uses specified weight matrix	Sharpening, edge enhancement, etc.

Larger filter kernels provide wider smoothing range but also increase computation and temporary array overhead.

Code Overview

Initialize Camera and Display Module

```
w, h = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=w, height=h)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_id=True)
sensor.run()
time.sleep(0.5)
```

Create Sharpening Kernel and Set Mode

```
sharpen = np.array([
    [-1.0, -1.0, -1.0],
    [-1.0, 9.0, -1.0],
    [-1.0, -1.0, -1.0]
])

mode = 0
```

The sharpening kernel has a center weight of 9 and the surrounding 8 positions are -1, totaling 1, so it enhances local differences while roughly preserving overall brightness.

\$mode\$	Function	Current Parameters
0	\$GaussianBlur\$	7×7, sigma auto, source default
1	\$blur\$	7×7
2	\$medianBlur\$	aperture 7
3	\$bilateralFilter\$	d=7, sigmaColor=50, sigmaSpace=50
4	\$boxFilter\$	7×7, normalized
5	\$filter2D\$	3×3 sharpening kernel

Execute Filtering

```
if mode == 0:
    out = cv2.GaussianBlur(img_np, (7, 7), 0)
elif mode == 1:
    out = cv2.blur(img_np, (7, 7))
elif mode == 2:
    out = cv2.medianBlur(img_np, 7)
elif mode == 3:
    out = cv2.bilateralFilter(img_np, 7, 50, 50)
elif mode == 4:
    out = cv2.boxFilter(img_np, -1, (7, 7), normalize=True)
else:
    out = cv2.filter2D(img_np, -1, sharpen)
```

ddepth=-1 means the output depth matches the input. Out-of-range results in 8-bit images are handled according to API rules. Pay attention to highlight and dark area saturation when sharpening.

Write Back to Original Frame and Display

```
img_np[:, :, :] = out
Display.show_image(img)
```

The output array is written back to the shared frame for display. If the original image is still needed afterwards, complete related calculations or explicitly copy the required areas before overwriting.

Garbage Collection and Resource Release

```
if fc % 10 == 0:
    gc.collect()

finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Kernel Size:** Start with 3×3 or 5×5 for Gaussian, mean, and box filters. 7×7 is the current example value.
- **Median Aperture:** Should be an odd number greater than 1; larger aperture significantly increases computation.
- **Bilateral Filter:** Larger sigmaColor means pixels with greater color differences also affect each other; sigmaSpace controls the spatial neighborhood range.
- **Sharpening Strength:** The current kernel is relatively strong and will also amplify noise. You can reduce the center weight while keeping the kernel weight sum close to 1.
- **Frame Rate Comparison:** Bilateral and median filtering are usually slower than mean and Gaussian filtering. Test under the same scene and display settings.
- **Edge Detection Preprocessing:** When you need to reduce noise while preserving edges as much as possible, start with small-kernel Gaussian filtering rather than directly using large-kernel blurring.